Using the AIX Lab-Atlas skills

in Claude Code

Version	2
For	Designers using Claude Code in the Claude desktop app. No terminal needed. Setup takes about 10 minutes, once.
Source	https://github.com/RPalis/AIXLab-Atlas. This is a team guide, not an official Anthropic document.
Use	Internal. Do not share outside the team.

What you are setting up

The repository holds 30 skills. Each skill is a folder with a `SKILL.md` file of instructions, for example `audit`, `brief` and `persona`. You copy those folders to one place on your computer. After that, Claude Code can run any skill by name, in any project.

Step 1. Get the files

1. Open https://github.com/RPalis/AIXLab-Atlas and sign in to GitHub with the account that was invited.
2. Click the green **Code** button, then **Download ZIP**. Unzip it: on a Mac double-click the file; on Windows right-click it and choose **Extract All**.

> **CHECK** The unzipped folder contains a folder called `skills`.

Step 2. Copy the skills

1. Open the Claude desktop app (claude.com/download if you do not have it), sign in, open the **Code** tab once, then quit the app. This creates the folder you need.
2. Open your `.claude` folder. **Mac:** in Finder press **Cmd+Shift+G**, type `~/.claude`, and press Enter. **Windows:** press **Win+R**, type `%USERPROFILE%\.claude`, and press Enter.
3. If there is no folder called `skills`, create one and name it `skills` in lowercase (Mac: **File > New Folder**. Windows: right-click, **New > Folder**).
4. Open the unzipped download, open its `skills` folder, select all 30 folders (**Cmd+A** on a Mac, **Ctrl+A** on Windows), and drag them into the `skills` folder from step 3. If you are asked about folders that already exist, choose to replace them.

> **CHECK** The `skills` folder inside `.claude` holds 30 folders, and each one contains a file called `SKILL.md`.

Step 3. Check that Claude Code sees them

1. Open the Claude desktop app and the **Code** tab. Choose your project folder as the working folder.
2. Type `/` and look at the list.

> **CHECK** The list includes skills such as `setup`, `brief`, `persona` and `audit`. If it shows none, quit the app completely, reopen it, and check Step 2.

> **NOTE** `/help` is also a built-in Claude Code command, and the built-in wins. To run this suite's help, type: Use the AIX Lab-Atlas help skill. Every other skill name works as a slash command.

Step 4. Use a skill

Type a slash, the skill name, and an optional mode or idea, then send. Not sure where to start? Ask for the help skill. It reads your project's files and recommends the next one to three skills.

```
/brief an app to share weekly meal plans with housemates
/audit craft
Use the AIX Lab-Atlas help skill
```

The slash is optional: "Run the audit skill in craft mode" does the same thing. A skill that needs a mode asks for it if you do not give one.

Phase	Commands (modes in brackets)
Before Discover	`/setup`, `/brief`
Discover	`/workshop` [plan, ingest], `/competitor-analysis` [discover, add, analyse], `/ux-benchmark`, `/interview`, `/interview-analysis`, `/persona` [card, day], `/journey` [map, blueprint, pain-points, solutions, experience], `/problem-statement`, `/metrics`
Decide	`/brainstorming`, `/information-architecture`, `/map` [objects, site, flow, audit, document], `/interrogate`, `/decisions`
Design	`/prototype`, `/styleguide` [default, forge], `/audit` [craft, heuristic, code, spec, a11y], `/refine`, `/storybook`, `/figma` [capture, import, sync]
Deliver	`/usability-test-plan`, `/package` [full, lite], `/showcase`, `/guide`, `/requirements`
Any time	`/wiki` [compile, note], `/skill-manager`

Audit modes. `craft` scores five dimensions 0 to 4 and ranks a task list P0 to P3. `heuristic` is a 12-point usability pass for when the audit has to stand as evidence. `a11y` checks WCAG 2.1 and 2.2 AA. `code` checks token use and structure. `spec` checks the build against the approved design. Audit only finds problems; run `/refine` to fix them. Never average craft and heuristic scores.

Rules

- **Plan first.** Turn on plan mode (the mode selector near the prompt box, or Shift+Tab). Claude shows a plan and waits for your yes before it changes anything.
- **No invented evidence.** No made-up people, quotes, numbers or competitors. If something is missing, it goes on a line in `UNKNOWNS.md` and Claude asks you. A guess must be labelled as a guess.
- **Name the source** of every finding: the file, the screen, or your words.
- **Stay in scope.** Practise in a folder of your own with an invented idea, never client material. Never paste passwords, keys or tokens into a chat. Keep Claude on its default, which asks before editing.

- **Never claim what did not happen.** A skill must not say it wrote a file or ran a command when it did not.

Paste this before a skill run to enforce them:

```
Use only files in this project and the skill's own SKILL.md. Do not invent users, quotes,
numbers, competitors or screens. If something is missing, add it to UNKNOWNS.md and ask m
e.
Show me a plan first and wait for my yes before creating or changing files. Name the sourc
e
file for every finding. If you guess, say it is a guess.
```

How to: design systems

Three jobs come up again and again: connect a design system that already lives in Git, create a new one with foundations only, and (in Claude Design) bring in a component library and style guide from Figma. **Foundations** means the base tokens: colour, typography, spacing, radius and shadow, in light and dark. It does not include components.

1. Connect a design system from Git

Pick the case that matches your repository.

Case A: it is a React component library and you want Claude Design to use it.

1. Download the repository as a ZIP (green **Code** button, then **Download ZIP**) and unzip it. For a private repository, sign in with an invited account first.
2. Open the Claude desktop app, open the **Code** tab, and choose the unzipped folder as the working folder.
3. Type `/design-sync` and press Enter. Follow the prompts. On a large library it can take a long time and may ask you to confirm the cost. Only confirm if you accept it.
4. Open claude.ai/design, go to **Settings > Design systems**, and check the new system is listed. Turn on **Published** so new projects use it.

> **CHECK** The system appears under **Settings > Design systems**. `/design-sync` works with React libraries. For other code, use the Claude Design route in the Claude Design sheet.

Case B: you want the tokens documented in this suite, whatever the code is.

1. Unzip the repository and choose its folder in the **Code** tab, as above.
2. Type `/styleguide` and send: `Audit and document the design tokens in this project. Do not change any value.`
3. Read the `DESIGN.md` it writes. Anything it could not work out from the code goes on a line in `UNKNOWNS.md` and Claude asks you.

> **CHECK** `DESIGN.md` exists, and every value in it points to a file in the repository. Nothing was changed.

2. Create a design system (foundations only)

1. Decide: do you have a real brand? If not, `/styleguide` builds a placeholder palette, labelled as placeholder. If yes, give it your sources: a live website, a brand guide (PDF, Figma or document) or hex codes you type in.

2. In the **Code** tab, type `/styleguide` and send: `Foundations only: colour, typography, spacing, radius and shadow, light and dark. No components.` For a real brand add: `Use forge mode.`
3. Answer its questions. If two sources disagree, for example the guide says one blue and the website shows another, it asks which wins. Tints it works out itself are labelled "computed, not brand-verified".
4. It writes `src/index.css` (the raw brand palette plus the semantic roles, light and dark), the Tailwind mapping and `DESIGN.md`. With no project yet, tell it so. It then returns the token block and the `DESIGN.md` text in the conversation, and you save them as files.

> **CHECK** `index.css` has one variable for every role in the skill's role list, in light and dark, and a search for hard-coded colours outside `index.css` finds none.

3. Figma library to tokens

`/figma` in mode `import` or `sync` reads a Figma library through the Figma connector and writes its values into `index.css` and `DESIGN.md`, asking before it overwrites anything. The connector is set up once. Ask the AIX-Lab support channel to do that for you. To bring a Figma component library and style guide into Claude Design itself, use the Claude Design sheet.

If something goes wrong

What you see	What to try
Typing `/` shows no skills	Quit Claude completely and reopen it. Check that your `.claude/skills` folder holds 30 folders.
`/help` opens Claude Code's own help	Type: Use the AIX Lab-Atlas help skill.
A skill writes before showing a plan	Type: stop, show me your plan first.
A skill says it cannot write a file or run a command	Take the output it returns in the conversation. Skills that build a prototype need a shell and may say so.